Q5: QPS 300 → 3,000 的擴展策略

先說明

老實說，我的經驗主要在前端，沒有直接操作 AWS EC2、MySQL DBA 或 Laravel 後端架構的實務經驗。但在 Nidin 做過日活 120 萬用戶的產品，對高併發場景有一些理解。我習慣先把問題拆開來看，再決定從哪邊下手。

我會怎麼做

1. 前端：減少不必要的 API 請求

這是我最能直接處理的部分，也是我會先動手的地方：

- 靜態資源（JS、CSS、圖片）上 CDN，減輕 EC2 的流量壓力
- API response 加上適當的 cache header，讓瀏覽器快取不常變動的資料
- 前端加 debounce / throttle，避免短時間內重複打 API
- 合併可以一起拿的 API call，減少 request 數量

2. 加 Redis 快取層

概念上理解，但我沒有實際部署過 Redis server：

- 在 Laravel 跟 MySQL 之間加一層 Redis cache
- 把熱門查詢、不常更新的資料快取起來
- 大部分讀取請求不需要打到 DB，直接從 cache 回

QPS 從 300 到 3,000，如果大部分是讀取，cache hit rate 高的話應該能撐住。

3. DB 層優化

- 確認常用查詢欄位有建 index
- 用 `EXPLAIN` 分析慢查詢，看有沒有 full table scan
- 如果讀寫比例懸殊，考慮加 Read Replica 分擔讀取壓力

4. 應用層

- 現有 2 台 EC2，確認前面有沒有 Load Balancer 分流

- 如果沒有，加一台 ALB 把流量分散到兩台
- Laravel 的 session 和 cache 改用 Redis，這樣兩台 EC2 才能共享狀態

優先順序

在 2~3 天又不能停機的限制下，我會這樣排：

1. **前端 CDN + cache header**（半天）→ 我能直接做，馬上減少 EC2 負擔
2. **Redis cache**（一天）→ 效果最明顯，需要後端同事配合
3. **DB index 優化**（半天）→ 需要 DBA 或後端確認
4. **Load Balancer**（半天）→ 需要有 AWS 權限的人處理

會這樣排是因為前端的部分風險最低、我最有把握，先做完再往後端推。

小結

這題有些我能直接動手（前端優化、CDN），有些需要跟後端和 DevOps 協作（Redis、DB、Load Balancer）。我沒辦法一個人完成所有項目，但我的做法是先把自己能做的做掉，確認有效果之後，再跟團隊溝通後端的優化方向。